

ETF 追蹤器

模組拆分架構說明 v2

生成日期：2026-06-30 已完成所有模組拆分

一、專案背景

原始 app.py 為 2,972 行的單體式 Flask 後端，包含資料庫連線、爬蟲邏輯、API 路由、AI 選股、背景執行緒等所有功能。本次依功能職責拆分為 6 個模組，完成後 app.py 縮減至 1,394 行（減少 53%），各模組獨立可測試、可維護。

二、最終目錄結構

```
etf-tracker-main/
├── app.py # 1,394 行 ETF API + 其他
├── db.py # DB init_db
├── scrapers/ #
│   ├── __init__.py
│   ├── holdings.py # MoneyDJ, etfinfo, pocket, FALLBACK
│   ├── prices.py # Yahoo Finance, TWSE, DB
│   └── changes.py # etfinfo active, DB
├── routes/ # API /
│   ├── __init__.py
│   ├── auth.py # API 10
│   ├── ai_stocks.py # AI 3 個 +
│   └── pages.py # /, /login, /admin
├── static/ # HTML
│   ├── index.html #
│   ├── login.html #
│   ├── admin.html #
│   └── ai_stocks.html # AI
├── db.py
├── requirements.txt
└── render.yaml
```

三、模組拆分對照表

模組	檔案	主要內容	搬移前行數	狀態
db-agent	db.py	DATABASE_URL、get_db()、init_db() 建立 8 張資料表 + 預設帳號	~180 行	完成
scraper-agent	scrapers/ holdings.py prices.py changes.py	持股爬蟲（MoneyDJ→etfinfo→pocket） ETF/個股股價（Yahoo+TWSE+DB快取） 日報爬蟲+DB快照	~530 行	完成
auth-agent	routes/auth.py	登入/註冊/改密/重設密碼 持倉 CRUD、主題儲存 （10 個 /api/auth*, /api/portfolio, /api/theme）	~210 行	完成

ai-stocks-agent	routes/ai_stocks.py	AI 候選股清單 (15 檔) 基本面爬蟲 (Yahoo V10 quoteSummary) 每週精選排程 (ROE+EPS+營收加權) 3 個 API 路由	~320 行	完成
frontend-agent	routes/pages.py static/*.html	/, /login, /admin 頁面路由 4 個 HTML 移至 static/ 目錄 ai_stocks.html 路由已在 ai_stocks.py	~40 行路由 +HTML 整理	完成

四、各模組詳細說明

4.1 db.py — 資料庫模組

管理所有 PostgreSQL 連線與 Schema 初始化，app.py 透過 `from db import get_db, init_db` 取用。
`init_db()` 自動建立 8 張資料表：

資料表	用途
users	帳號密碼 (bcrypt hash)
portfolios	使用者持倉 (ETF代號、持有張數、均價)
themes	使用者 UI 主題偏好 (JSON)
etf_configs	動態 ETF 清單 (可透過管理 API 新增)
etf_changes_history	每日操作日報快照 (加碼/減碼/新增/刪除)
stock_close_cache	個股收盤價 DB 快取 (減少 TWSE API 呼叫)
ai_stock_picks	每週 AI 題材股精選 (週一至日)
ai_stock_data	AI 候選股基本面資料 (EPS、ROE、PE)

4.2 scrapers/ — 爬蟲模組包

holdings.py (持股爬蟲，優先順序)：

1. MoneyDJ Basic0007B → 2. etfinfo.tw NUXT_DATA → 3. pocket.tw → 4. 投信官網 → 5. FALLBACK 靜態備援

prices.py (股價爬蟲)：

- `scrape_moneydj_price()`：Yahoo Finance V8 → TWSE STOCK_DAY，回傳現價/昨收/漲跌幅
- `get_stock_close()`：個股指定日期收盤 (DB快取 → TWSE → Yahoo)，供成本計算使用

changes.py (日報爬蟲)：

- `scrape_daily_changes()`：抓取 etfinfo.tw/active 加碼/減碼/新增/刪除明細
- `save_changes_snapshot()`：UPSERT 至 etf_changes_history，避免重複儲存

4.3 routes/auth.py — 認證、持倉、主題 API

使用 Flask Blueprint (auth_bp)，包含 10 個路由：

路由	方法	功能
/api/auth/login	POST	登入，設定 30 天 session
/api/auth/register	POST	註冊新帳號（密碼最少 6 字元）
/api/auth/logout	POST	清除 session
/api/auth/me	GET	取得目前登入帳號
/api/auth/change_password	POST	修改密碼（需驗舊密碼）
/api/auth/reset_password	POST	重設密碼（需管理員密碼）
/api/portfolio	GET	取得持倉資料
/api/portfolio	POST	儲存持倉資料（UPSERT）
/api/theme	GET	取得 UI 主題
/api/theme	POST	儲存 UI 主題

4.4 routes/ai_stocks.py — AI 題材股精選

使用 Flask Blueprint (ai_stocks_bp)，包含資料邏輯 + 3 個路由 + 背景排程執行緒。

函式/路由	說明
AI_STOCK_CANDIDATES	15 檔 AI 題材候選股清單（含 tags）
fetch_stock_fundamentals()	Yahoo V8 + quoteSummary 抓取 EPS/ROE/PE/淨利率
save_ai_stock_data()	UPSERT 至 ai_stock_data 資料表
refresh_ai_stock_picks()	依 $ROE \times 0.4 + EPS \text{成長} \times 0.4 + \text{營收成長} \times 0.2$ 排序，取前10
_ai_stock_weekly_scheduler()	背景執行緒：週五 18:00 台灣時間自動更新
GET /api/ai-stocks	取得本週精選清單（1 小時 cache）
POST /api/ai-stocks/refresh	手動觸發更新（需登入）
GET /AI_Stocks 或 /ai-stocks	回傳 static/ai_stocks.html

4.5 routes/pages.py + static/ — 頁面路由與靜態資源

使用 Flask Blueprint (pages_bp)，讀取 static/ 目錄下的 HTML 並回傳，需登入的頁面在路由層做 session 檢查。

路由	HTML 檔案	說明
/login	static/login.html	登入頁面（無需登入）
/	static/index.html	主頁面（需登入，自動替換 API URL）
/admin	static/admin.html	管理後台（需登入）
/AI_Stocks /ai-stocks	static/ai_stocks.html	AI 題材股頁面（ai_stocks.py 處理）

五、拆分成效總結

指標	拆分前	拆分後	變化
app.py 行數	2,972 行	1,394 行	減少 53%
模組數量	1 個檔案	9 個模組	+8 個
可獨立測試	否	是	各模組單獨可測
DB 連線邏輯	散落全檔	db.py 集中	集中管理
爬蟲備援層級	2 層	5 層	+3 層
前端頁面管理	散落 app.py	routes/pages.py + static/	結構清晰
AI 選股模組	混在 app.py	routes/ai_stocks.py	獨立可擴充

六、模組 import 依賴關係

```
app.py
■■■ from db import get_db, init_db, DATABASE_URL
■■■ from scrapers.holdings import safe_get, scrape_etfinfo, FALLBACK, ...
■■■ from scrapers.prices import scrape_moneydj_price, get_stock_close
■■■ from scrapers.changes import scrape_daily_changes, save_changes_snapshot
■■■ from routes.auth import auth_bp
■■■ from routes.ai_stocks import ai_stocks_bp, _ai_stock_weekly_scheduler
■■■ from routes.pages import pages_bp

routes/auth.py → db.get_db()
routes/ai_stocks.py → db.get_db()
routes/pages.py → (■■■■■■■ Flask + os)
scrapers/changes.py → db.get_db()■■scrapers.holdings.safe_get()
scrapers/prices.py → db.get_db()■■■ import■■■■■■■
scrapers/holdings.py→ (■■■■■■■ requests + bs4)
```